

Large Language Models for Software Vulnerability Detection: Current Capabilities, Limitations, and the Road Ahead – A Systematic Review

Abstract

Large language models have drawn considerable interest as tools for identifying software vulnerabilities, largely because of their ability to interpret code semantics in ways that go beyond traditional methods. This review brings together insights from six recent peer-reviewed studies published in IEEE and ACM venues between 2024 and 2025. These works explore prompting techniques, model scaling, dataset challenges, and real-world performance. Although some approaches show modest improvements over conventional static analysis—especially in recall for certain vulnerability types—the overall picture that emerges is one of limited reliability. The evidence suggests that today’s LLMs are not yet ready to serve as standalone detectors. Instead, they appear most useful when integrated into hybrid systems that combine automated analysis with traditional tools and human judgment.

Introduction

Software vulnerabilities remain one of the most persistent threats in modern computing. A single undetected flaw can lead to data breaches, system compromise, or widespread disruption, as seen in high-profile incidents involving operating systems, web browsers, and enterprise software. Traditional static and dynamic analysis tools, while valuable, often struggle with scalability, produce high numbers of false positives, and lack the deeper semantic understanding needed to catch subtle or context dependent flaws (Shimmi et al., 2024; Khare et al., 2025). In response, researchers have turned to large language models pre-trained on vast code repositories. These models offer a different approach, one grounded in contextual reasoning rather than rigid pattern matching.

This systematic review examines the current state of LLM-based vulnerability detection by focusing on six high-quality studies from leading IEEE and ACM conferences and journals: Shimmi et al. (2024), Khare et al. (2025), Lin and Mohaisen (2025), Ding et al. (2025), Jin (2025), and Sheng et al. (2025). Collectively, they evaluate dozens of models across multiple datasets and languages, test various prompting strategies, and scrutinize the role of dataset quality.

The review has three primary aims. First, it summarizes the main techniques being explored. Second, it compares reported capabilities against documented limitations. Third, it offers a clear, evidence based assessment of whether these models are practically effective today. The findings point to cautious optimism tempered by significant practical constraints. While LLMs demonstrate genuine strengths in semantic reasoning, the literature consistently shows that they fall short of the reliability required for production grade security workflows. This hybrid survey and review structure provides a balanced foundation for researchers and practitioners seeking to understand both the promise and the current boundaries of the technology.

Background

Khare et al. (2025) discuss how conventional vulnerability detection has long relied on rule-based static analyzers, fuzz testing, and classical machine learning classifiers. These methods excel at identifying known patterns but often falter when faced with novel or semantically complex issues. LLMs introduce a new dimension by treating code as natural language sequences, enabling zero-shot, few-shot, and fine-tuned evaluation modes.

Researchers typically assess performance using established datasets such as BigVul, CVEFixes, and newer, cleaner benchmarks like PrimeVul. Standard metrics include accuracy, precision, recall, and F1-score, with particular attention paid to class imbalance common in vulnerability data. Khare et al. (2025) describe the metrics in the context of vulnerability detection, shown in Figure 1.

Figure 1

Vulnerability detection metrics (Khare et al., 2025).

- **Accuracy:** Accuracy measures how often the tool correctly predicts whether a code snippet is vulnerable or not. It is computed as: $\frac{\text{True Positives} + \text{True Negatives}}{\text{\#Samples}}$.
- **Precision:** Precision measures the proportion of cases that a tool detects as a vulnerability that are indeed true vulnerabilities. It is computed as: $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$.
- **Recall:** Recall measures the proportion of all vulnerabilities that the tool can correctly detect. It is computed as: $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$.
- **F1 score:** The F1 score is a harmonic mean of precision and recall. It is computed as: $2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$.

Programming Languages in Datasets and Model Performance

Most benchmarks focus heavily on C/C++ and Java, with limited representation of other languages such as Python or JavaScript in early studies. Khare et al. (2025) explicitly evaluate both Java and C/C++ datasets, finding stronger performance on intra-procedural vulnerabilities common in these languages. Performance varies across languages, with some models showing better results in Java due to

more structured patterns in training data. This language skew limits generalizability claims and highlights the need for broader multilingual benchmarks. Ding et al. (2025) note that dataset construction often favors C/C++ functions from open-source repositories, which may not reflect the diversity of modern software ecosystems.

Dataset quality emerges as a recurring and critical concern. Many older corpora contain label noise, duplicated samples, and other artifacts that can produce overly optimistic results. For instance, Ding et al. (2025) highlight how such issues inflate reported performance and introduce data leakage risks. Evaluations typically occur at snippet or file level, with growing emphasis on repository scale analysis to reflect real-world conditions. Metrics must therefore account for both detection accuracy and practical usability, including the ability to distinguish vulnerable code from its patched counterparts.

Traditional tools such as CodeQL provide a useful baseline for comparison. They rely on manually crafted queries and project wide context, offering high precision in controlled environments but limited scalability for partial or incomplete code snippets. LLMs, by contrast, operate on pre-trained knowledge of APIs and coding patterns, potentially reducing the need for extensive manual specification. However, this advantage comes with trade-offs in consistency and interpretability. Understanding these foundational differences is essential before examining specific LLM techniques and empirical outcomes.

Techniques for LLM-Based Vulnerability Detection

Several promising techniques have emerged in recent work. Shimmi et al. (2024) examined whether supplying additional contextual information—such as function descriptions, security specifications, or a few labeled examples—improves detection accuracy. Their experiments with designs that pair source code with CWE descriptions—via bimodal CodeBERT embeddings or unimodal SBERT

Table 1

Big-Vul: Results from Basic Prompt (Shimmi et al., 2025).

TABLE 1: Big-Vul: Results from the Basic Prompt.									
Input	TP	TN	FP	FN	Acc.	Pre.	Rec.	F1	F2
Method	8,500	176	8,729	210	49.25%	49.33 %	97.59%	65.54%	81.62%

semantic matching—demonstrated clear gains over basic prompts. Table 1 shows, in zero-shot settings

Table 2

LLM accuracy using few-shot examples and domain specific knowledge (Shimmi et al., 2025).

TABLE 2: The accuracy of LLM in detecting method-level vulnerability with “bimodal-generated” and “unimodal-generated” domain information. The circled numbers are associated with circled numbers in Figure 2.										
Approach		TP	TN	FP	FN	Acc.	Pre.	Rec.	F1	F2
Bimodal										
Zero-Shot ②	Precision Focus	4,681	1,493	4,726	729	45.58%	49.76%	63.91%	55.96%	60.47%
Zero-Shot ②	Recall Focus	4,646	1,045	4,940	624	50.56%	48.47%	88.16%	62.55%	75.75%
Few-Shot ③	Precision	2,896	4,169	2,883	2,655	56.06%	50.11%	52.17%	51.12%	51.75%
Few-Shot ③	Recall	3,603	4,047	2,944	2,643	57.79%	55.03%	57.68%	56.33%	57.13%
Unimodal										
Zero-Shot ②	Precision Focus	4,829	1,049	5,394	702	49.09%	47.24%	87.31%	61.31%	74.64%
Zero-Shot ②	Recall Focus	4,927	937	5,373	563	49.69%	47.83%	89.74%	62.41%	76.37%
Few-Shot ③	Precision Focus	3,247	4,190	2,888	3,088	55.45%	52.93%	51.25%	52.08%	51.58%
Few-Shot ③	Recall Focus	3,369	3,873	2,905	2,576	56.92%	53.70%	56.67%	55.14%	56.05%

without extra information, models frequently defaulted to labeling code as vulnerable, achieving only about 49% accuracy with extremely high recall (97.59%) but poor precision. Introducing few-shot examples and domain specific knowledge raised accuracy to the 55–57% range, although this came with a modest tradeoff in recall as models became better at identifying true negatives, as seen in Table 2.

Khare et al. (2025) tested more structured prompting methods, including step-by-step reasoning chains that simulate dataflow analysis. These strategies produced meaningful gains in F1-scores, particularly on real-world datasets, with improvements of up to 0.18. Lin and Mohaisen (2025) conducted one of the broadest comparisons to date, evaluating 21 different models while varying parameters such as model size, quantization levels, and context window length. They found that larger context windows consistently helped performance across families, whereas quantization produced more mixed outcomes depending on the specific architecture.

Complementary surveys by Jin (2025) and Sheng et al. (2025) organize these developments into broader categories. They highlight trends toward retrieval augmented generation, fine-tuning strategies, and early multimodal designs that integrate code structure with natural language specifications. Together, these techniques represent a shift from treating LLMs as black-box classifiers to more deliberate systems that leverage external knowledge and structured reasoning.

Empirical Findings from Recent Studies

When taken together, the empirical results present a nuanced and sometimes conflicting picture. Khare et al. (2025) conducted a comprehensive evaluation of 16 models across multiple datasets and vulnerability types. As summarized in Table 3, the study reports an average accuracy of approximately 62.8% and a mean F1-score of 0.71 across all datasets.

Performance was notably stronger for intra-procedural vulnerabilities such as OS Command Injection and NULL Pointer Dereference. Figure 2 further illustrates the effectiveness of various LLMs

Table 3

Summary of research questions and key findings (Khare et al., 2025, Table 1).

TABLE I: Summary of our research questions and key findings	
Research Questions	Findings
RQ1: How do different pre-trained LLMs perform in detecting security vulnerabilities across different languages and datasets? (Section §III-A)	✓ LLMs across all sizes report a mean accuracy of about 62.8% and a mean F1 score of 0.71 across all datasets. ✗ Average accuracy on real-world datasets is 10.5% lower than that on synthetic datasets. ✓ In stark contrast to other domains, smaller models such as Qwen-2.5-14B and Qwen-2.5-32B report higher accuracies on the real-world datasets than much larger models such as GPT-4.
RQ2: How do different prompting strategies affect the performance of LLMs? (Section §III-B)	✓ Using prompts that focus on detecting specific CWEs improves the performance of LLMs. ✓ The Dataflow analysis-based prompt further improves results for larger LLMs with an increase of up to 0.18 in F1 score on real-world datasets. ✓ We also observe that LLMs often infer the correct sources/sinks/sanitizers but fail in end-to-end reasoning.
RQ3: How does the performance of LLMs vary across different vulnerability classes? (Section §III-C)	✓ LLMs are better at detecting local vulnerabilities that require no global context across datasets (OS Command Injection, NULL Pointer Dereference, Out-of-bounds Read/Write, etc.). ✗ LLMs struggle to detect vulnerabilities that require additional context or reasoning about complex data structures (Out-of-bounds Read/Write with C++ structs and pointers). ✓ Certain LLMs are very good at detecting specific CWEs across datasets (Llama-3.1-70B on OS Command Injection, DeepSeekCoder-7B on NULL Pointer Dereference, etc.).
RQ4: How do LLMs compare to state-of-the-art static analysis tools? (Section §III-D)	✗ LLMs report lower overall accuracies than CodeQL on all synthetic datasets. ✓ LLMs report higher accuracies than CodeQL on certain vulnerability classes across datasets (Path Traversal, OS Command Injection, etc). CodeQL reports higher accuracies on Integer Overflow across datasets.
RQ5: How do LLMs compare to state-of-the-art deep-learning-based tools? (Section §III-E)	✓ Deep Learning(DL)-based tools such as DeepDFA [10] and LineVal [11] report accuracies similar to Qwen-2.5-32B on CVEFixes C/C++ even after being trained on in-distribution samples whereas Qwen-2.5-32B reports higher F1 scores. ✓ DL-based tools report lower accuracies and F1 scores than LLMs when trained and evaluated on different datasets. ✓ LLMs provide natural language explanations for their predictions while DL-based tools provide binary scores and line numbers that are often difficult to interpret.

across Java and C/C++ datasets under different prompting strategies. Notably, certain smaller models (e.g., Qwen-2.5 variants) occasionally achieved comparable or higher performance than much larger models such as GPT-4, particularly on real-world datasets.

Figure 2

Effectiveness of LLMs in predicting security vulnerabilities in Java and C/C++ (Khare et al., 2025, Fig. 1).

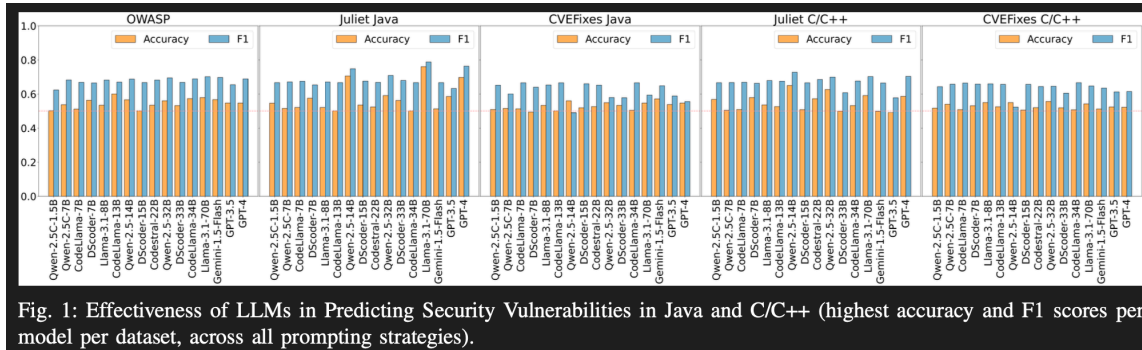


Fig. 1: Effectiveness of LLMs in Predicting Security Vulnerabilities in Java and C/C++ (highest accuracy and F1 scores per model per dataset, across all prompting strategies).

Shimmi et al. (2024) similarly noted gains from enriched prompts but highlighted difficulties in pure zero-shot settings. Lin and Mohaisen (2025) documented substantial variability: models such as Gemma and certain LLaMA-2 variants performed respectably in Java, with accuracy metrics reaching the high 70s in optimized configurations. However, many setups approached random guessing once realistic negative samples were included, and performance fluctuated significantly across languages and learning modes. Context window size emerged as a reliable positive factor, while quantization benefits proved inconsistent.

The most critical assessment comes from Ding et al. (2025). By introducing the cleaner PrimeVul benchmark—with rigorous labeling, deduplication, and chronological splitting—they showed that state-of-the-art models achieved respectable F1-scores on noisy legacy datasets like BigVul (e.g., 68.26% for StarCoder2) but collapsed dramatically to as low as 3.09% on higher-quality PrimeVul data (see Table 4). Even GPT-4 with chain-of-thought prompting struggled in pair-wise evaluations that required distinguishing vulnerable code from its benign counterpart. Table 4

Table 4

Performance across various LLMs (Ding et al., 2025, Table 6).

TABLE VI: Performance of code-LM-based vulnerability detection models in different settings.

Model	Train	Test	Acc ↑	F1 ↑	VD-S ↓	P-C ↑	P-V ↓	P-B ↓	P-R ↓
CTS	BV	BV	95.67	64.93	77.30	24.98	50.90	22.79	1.33
	PV	PV	97.00	5.82	95.97	0.18	3.01	96.10	0.71
	PV	PV	96.67	19.7	89.93	1.06	12.94	84.75	1.24
CB	BV	BV	95.57	62.88	81.77	22.60	48.34	27.83	1.23
	PV	PV	97.04	4.49	95.54	0.35	1.95	96.99	0.71
	PV	PV	96.87	20.86	88.78	1.77	11.35	86.17	0.71
UC	BV	BV	96.46	65.46	62.30	39.60	23.74	33.24	3.42
	PV	PV	97.27	1.94	95.11	0.35	0.35	98.76	0.53
	PV	PV	96.86	21.43	89.21	1.60	12.06	85.11	1.24
SC2	BV	BV	96.20	68.26	69.14	35.23	41.98	20.61	2.18
	PV	PV	97.09	3.09	96.83	0.89	0.89	97.70	0.53
	PV	PV	97.02	18.05	89.64	2.30	8.16	88.30	1.24
CG2.5	BV	BV	96.57	67.30	61.73	40.84	26.02	29.63	3.51
	PV	PV	97.23	1.91	95.68	1.24	0.00	98.76	0.00
	PV	PV	96.65	19.61	91.51	3.01	10.82	84.22	1.95

CTS: CodeT5, CB: CodeBERT, UC: UnixCoder, SC2: StarCoder2, CG2.5: CodeGen2.5, BV: BigVul, PV: PrimeVul, VD-S is the Vulnerability Detection Score defined as false negative rate (FNR) @ false positive rate (FPR) $\leq 0.5\%$ in Section IV-B1, P-C, P-V, P-B, and P-R are the metrics defined in Section IV-B2 to evaluate the models on paired functions.

illustrates this contrast. Jin (2025) and Sheng et al. (2025) synthesize these observations, noting incremental progress in prompting and scaling alongside enduring challenges in generalization and interpretability.

Capabilities and Promising Directions

Despite the limitations, LLMs do show genuine strengths. They excel at semantic reasoning and can surface subtle issues that purely syntactic tools often miss, particularly for intra-procedural vulnerabilities in C/C++ and Java. Well-designed prompts and additional contextual information can meaningfully boost recall for certain vulnerability classes. Larger context windows and carefully selected model families also appear beneficial in controlled evaluations. These capabilities suggest LLMs could serve as valuable augmentative components, helping human analysts focus attention on higher risk areas of code. Surveys by Jin (2025) and Sheng et al. (2025) further point to hybrid integrations—such as retrieval-augmented generation or multimodal architectures—as promising avenues for future refinement.

Critical Limitations and Practical Gaps

Significant barriers to practical deployment remain despite the observed capabilities. Overall performance stays modest and is highly sensitive to prompt design, model choice, dataset characteristics, and configuration parameters. Lin and Mohaisen (2025) provide detailed insight into

Table 5

Tested models with parameters, quantization, and context window (Lin & Mohaisen, 2025, Table I).

TABLE I: Tested models. ★ is the mode’s mini-version.

Model	Parameters	Version	Type	Quantization	CW
LLaMA-2	7B	-	Chat	q5_K_M	4096
LLaMA-2	13B	-	Chat	q5_K_M	4096
LLaMA-2	70B	-	Chat	q5_K_M	4096
CodeLLaMA	7B	-	Instruct	q5_K_M	16384
CodeLLaMA	34B	-	Instruct	q5_K_M	16384
CodeLLaMA	70B	-	Instruct	q5_K_M	2048
LLaMA-3	8B	-	Instruct	q5_K_M	8192
LLaMA-3	70B	-	Instruct	q5_K_M	8192
Mistral	7B	v0.2	Instruct	q5_K_M	32768
Mistral	8x7B	v0.1	Instruct	q5_K_M	32768
Gemma	2B	v1.1	Instruct	q5_K_M	8192
Gemma	2B	v1.1	Instruct	fp16	8192
Gemma	7B	v1.1	Instruct	q5_K_M	8192
Gemma	7B	v1.1	Instruct	fp16	8192
CodeGemma	7B	v1.1	Instruct	q5_K_M	8192
CodeGemma	7B	v1.1	Instruct	fp16	8192
Phi-2	2.7B	v2	Chat	q5_K_M	2048
Phi-2	2.7B	-	Chat	fp16	2048
Phi-3	3.8B★	-	Instruct	q5_K_M	4096
Phi-3	3.8B★	-	Instruct	fp16	4096
GPT-4	-	-	Chat	-	-

these

Table 6

Performance comparison of various LLM models showing precision (P), recall (R), and F1 scores (Lin & Mohaisen, 2025, Table III).

TABLE III: Performance comparison of various LLM models for Java and C/C++ vulnerability detection with positive-negative samples using precision, recall, and F1 score. The total samples for Java were 280 and for C/C++ were 200 samples. ¹ mini-4k-instruct.

MF	Param	Quant	CW	Java			C/C++		
				P	R	F1	P	R	F1
LLaMA-2	7B	q5_K_M	4096	41.38	68.57	51.61	20.16	25.00	22.32
LLaMA-2	13B	q5_K_M	4096	37.43	45.71	41.16	8.08	8.00	8.04
LLaMA-2	70B	q5_K_M	4096	41.18	70.00	51.85	20.80	26.00	23.11
CodeLLaMA	7B	q5_K_M	16384	51.61	22.86	31.68	28.57	32.00	30.19
CodeLLaMA	34B	q5_K_M	16384	24.68	13.57	17.51	8.05	7.00	7.49
CodeLLaMA	70B	q5_K_M	2048	28.95	39.29	33.33	9.09	10.00	9.52
LLaMA-3	8B	q5_K_M	8192	42.86	38.57	40.60	20.56	22.00	21.26
LLaMA-3	70B	q5_K_M	8192	23.53	2.86	5.10	0.00	0.00	0.00
Mistral	7B	q5_K_M	32768	50.88	20.71	29.44	20.51	8.00	11.51
Mistral	8x7B	q5_K_M	32768	50.00	12.86	20.45	13.64	3.00	4.92
Gemma	2B	q5_K_M	8192	40.68	68.57	51.06	20.63	26.00	23.01
Gemma	2B	fp16	8192	44.35	78.57	56.70	21.43	27.00	23.89
Gemma	7B	q5_K_M	8192	44.50	69.29	54.19	29.29	41.00	34.17
Gemma	7B	fp16	8192	46.19	77.86	57.98	29.58	42.00	34.71
CodeGemma	7B	q5_K_M	8192	65.38	48.57	55.74	23.71	23.00	23.35
CodeGemma	7B	fp16	8192	54.05	42.86	47.81	20.79	21.00	20.90
Phi-2	2.7B	q5_K_M	2048	29.08	40.71	33.93	9.91	11.00	10.43
Phi-2	2.7B	fp16	2048	28.72	40.00	33.43	9.91	11.00	10.43
Phi-3	3.8B ¹	q5_K_M	4096	26.58	30.00	28.19	6.93	7.00	6.97
Phi-3	3.8B ¹	fp16	4096	23.40	23.57	23.49	4.12	4.00	4.06

limitations through a large scale comparative study. Table 5 lists the 21 tested models along with their parameter sizes, quantization methods, and context window lengths. Table 6 presents the precision, recall, and F1 scores for Java and C/C++ across these configurations, revealing that many models achieve low practical utility, particularly on realistic positive-negative sample mixes.

Discussion and Overall Assessment

Current LLMs show modest effectiveness in controlled settings but fall short in practical, rigorous evaluations. The heavy focus on C/C++ and Java further limits broader applicability. While prompting and context augmentation provide incremental gains, the sharp drops on cleaned datasets underscore the gap between benchmark optimism and real-world demands.

On the basis of the reviewed evidence, current large language models are not yet effective as standalone solutions for practical software vulnerability detection. While they demonstrate promising semantic capabilities and can improve recall in narrowly defined settings, the empirical record—particularly the sharp performance drops observed on cleaner datasets—exposes persistent gaps in precision, generalization, and trustworthiness. Hybrid approaches that combine LLMs with traditional static analysis and careful human review currently offer the most realistic path forward. Continued advances in dataset quality, benchmarking standards, and multimodal integration will be necessary before these models can assume a more central role in security critical workflows.

Challenges, Future Directions, and Emerging Trends

Beyond the technical limitations already noted, researchers must also address challenges related to model interpretability, energy consumption, cross-language performance, and responsible use given the dual use nature of vulnerability discovery tools. Future work should prioritize the development and adoption of higher quality benchmarks, exploration of multimodal architectures that integrate code structure with natural language specifications, and creation of standardized evaluation protocols that better reflect real-world development environments. Jin (2025) and Sheng et al. (2025) emphasize the value of such efforts in moving the field toward more robust and deployable solutions.

Emerging trends outside the core peer-reviewed literature also merit attention. For example, Anthropic’s Claude Mythos Preview (2026) has demonstrated frontier capabilities in autonomous zero-day discovery and exploitation. As shown in Figure 3, Mythos Preview achieved a 72.4% success rate in

generating working JavaScript shell exploits for vulnerabilities in Firefox's JS engine — a dramatic improvement over previous models (Sonnet 4.6 and Opus 4.6). The benchmark results shown in Figure 3, highlight the rapid advancement in agentic cybersecurity capabilities. While such offensive prowess is currently restricted to vetted defensive use cases through initiatives like Project Glasswing, it underscores both the promise and the dual use risks of frontier LLMs.

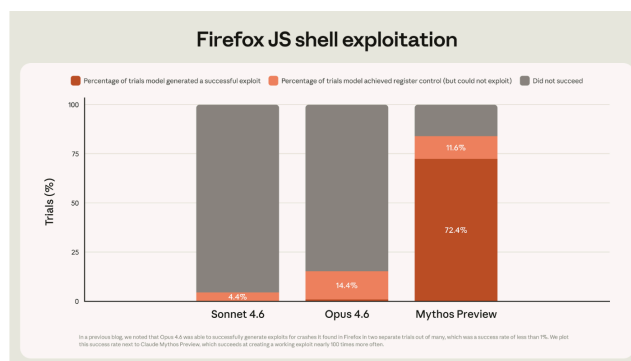
Conclusion

This review of six recent high quality studies illustrates both the rapid evolution and the current constraints of LLM-based vulnerability detection. The technology holds clear potential, yet practical deployment demands caution and integration with established methods. As the field matures, thoughtful combination of automated capabilities with human expertise will likely yield the greatest improvements in software security.

Ultimately, while frontier models continue to demonstrate impressive capabilities in controlled and agentic settings, the peer-reviewed evidence underscores the importance of tempered expectations for production environments. Researchers and practitioners should prioritize the development of robust hybrid pipelines and higher quality evaluation benchmarks to bridge the existing gap between promising research outcomes and dependable real-world security applications. Only through sustained, rigorous investigation can the full potential of large language models be safely and effectively realized in the critical domain of software vulnerability detection.

Figure 3

Firefox JS shell exploitation success rates for different Claude models (Anthropic, 2026).



References

- Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., & Chen, Y. (2025). Vulnerability detection with code language models: How far are we? In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE 2025)* (pp. 1729–1741). IEEE/ACM. <https://doi.org/10.1109/ICSE55347.2025.00038> (arXiv: <https://arxiv.org/abs/2403.18624>)
- Jin, S. (2025). Large language models for vulnerability detection in static code analysis: A survey. In *2025 8th International Conference on Artificial Intelligence, Big Data and Computing (ICAIBD)*. IEEE.
- Khare, A., Dutta, S., Li, Z., Solko-Breslin, A., Alur, R., & Naik, M. (2025). Understanding the effectiveness of large language models in detecting security vulnerabilities. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE. <https://www.cis.upenn.edu/~alur/ICST25.pdf>
- Lin, J., & Mohaisen, D. (2025). From large to mammoth: A comparative evaluation of large language models in vulnerability detection. In *Network and Distributed System Security (NDSS) Symposium 2025*. <https://www.ndss-symposium.org/wp-content/uploads/2025-1491-paper.pdf>
- Sheng, Z., Chen, Z., Gu, S., Huang, H., Gu, G., & Huang, J. (2025). LLMs in software security: A survey of vulnerability detection techniques and insights. *ACM Computing Surveys*, 58(5), Article 134. <https://doi.org/10.1145/3769082> (arXiv: <https://arxiv.org/abs/2502.07049>)
- Shimmi, S., Saini, Y., Schaefer, M., Okhravi, H., & Rahimi, M. (2024). Software vulnerability detection using LLM: Does additional information help? In *2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)*. IEEE. <https://doi.org/10.1109/ACSACW65225.2024.00031> (PDF: <https://web.mit.edu/ha22286/www/papers/WAITI24.pdf>)
- Anthropic. (2026). *Assessing Claude Mythos Preview's cybersecurity capabilities*. <https://red.anthropic.com/2026/mythos-preview/>